Soutenance 2

Maxime Outteryck, Lazare Nogrette, Thomas Pinglot, Jimmy Tran

April 2020

Contents

1 Introduction **3**

2 Mouvements et Plateformes **4**
2.1 clarté du code 4
2.2 Plateforme de Transition 5
2.3 Plateforme de saut 6

3 Level Design **8**
3.1 Idée du 1er niveau 8
3.2 Code de la balance 9
3.3 Influence du personnage 11
3.4 Objectifs 12

4 Site web **13**
4.1 Html et CSS 13
4.2 Hébergement du site 15

5 Menu **16**
5.1 MenuPause 16

6 Grappin **18**
6.1 Réalisation 18
6.1.1 Méchanisme du grappin 18
6.1.2 Déplacement du joueur 20
6.2 Tâches accomplies et à accomplir 21

7 Conclusion **21**

1 Introduction

Automatour est un jeu d'énigme en 2D dont la méchanique principale est un grappin qui permet au personnage principal de s'accrocher et se diriger vers des zones bien définies. Le jeu repose sur une utilisation optimale de cet objet et sur des principes physiques afin de pouvoir progresser.

Depuis la première soutenance, nous avons comme objectifs de répondre aux nouvelles attentes, qui sont principalement la mise en place d'un site dédié au jeu et la mise en commun de nos parties. Nous devions également améliorer notre code, l'adapter à l'idée que nous avons du rendu du jeu final de manière à obtenir quelque de jouable et cohérent, et avancer dans la conception général du jeu.

Nos rôles respectifs sont assez similaires à ceux que nous avons précédement. Maxime s'occupe des mouvements du personnage et d'autres aspects tels que certaines plateformes et le changement de scène, Jimmy a pour charge de s'occuper des différents outils du personnages, Thomas avait cette fois-ci la tâche de réaliser le site consacré au jeu, et Lazare doit imaginer les différentes énigmes et faire en sorte que la physique du jeu soit conforme aux besoins des niveaux et à leur bon fonctionnement.

Lors de la première soutenance, il nous a été reproché d'avoir un code un peu bordélique et globalement insuffisant en terme de quantité. Le simple fait d'avancer sur le projet et d'adapter notre code constitut une étape à la résolution de ce problème. Notre Git était également un peu défectueux, ce qui constituait un problème majeur pour la mise en commun de nos travaux respectifs. Bien que quelques interrogations subsistent, nous avons été capable d'échanger nos parties sans trop de difficultés cette fois-ci. Par ailleurs, puisque nous devions être aptes à répondre aux nouvelles attentes, à la fois celles de cette soutenance et du fonctionnement de notre jeu, nous pouvons affirmer assez sereinement que nous sommes devenus plus familiers avec le logiciel qui sert de moteur pour notre jeu : Godot, dont nous continuons de découvrir les avantages, mais également les faiblesses.

En sachant tout ceci, voici nos progrès ainsi que nos différentes procédures pour avancer dans la conception d'Automatour.

2 Mouvements et Plateformes

2.1 clarté du code

Mon premier objectif fut de réorganiser mon code car il n'était pas assez clair. Il était constitué d'une seule boucle principale où toutes les lignes de commandes se trouvaient les unes après les autres, pour vous donner une idée cela ressemblait à ça :

```
 public override void _Process(float delta)
{
      acc.x = 0;
      acc.y = gravity;

  if (Input.IsActionPressed("ui_right"))
  {
            if (OnGround)
            {
            acc.x = acceleration;
            }
            else
            {
                  acc.x = JumpAcceleration;
            }
  }

  if (Input.IsActionPressed("ui_left"))
  ...// et cela continue pour 120 lignes
```

Pour Le rendre plus clair j'ai divisé en 3 parties différentes ces lignes de commandes, une qui gère Les mouvements horizontaux, une autre Les mouvements verticaux ainsi que la gravité et une troisième qui s'occupe de gérer les animations du personnage, voici ce que cela donne :

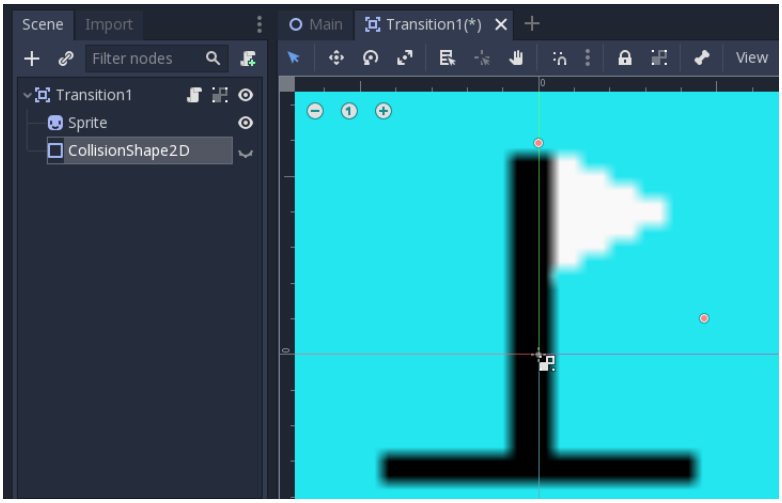
```
public override void _Process(float delta)
{
      Move();
      gravityExistDealWithIt();
      animeIsLife();
  motion = MoveAndSlide(motion, FLOOR);
}
```

Cela aide à la compréhension du Code et accélère le processus de modification puisqu'i n'y plus à chercher la ligne que l'on souhaite modifier parmis toutes les autres.

2.2 Plateforme de Transition

Mon second objectif était de créer le point d'arrivé du niveau à l'image des drapeaux dans les Mario bros lorsqu'on atteint ce drapeau, le niveau actuel s'arrête pour que l'on puisse accéder au niveau suivant. Pour cela j'avais besoin de détecter lorsque le joueur entrait en contact avec le drapeau. Ce fut le défi de cette partie la détection de collision.

Pour faire cela Godot offre un outil pratique qui est L'area2D. Lorsqu'on créer un nouvel élément sur Godot on a le choix entre un kinematicBody, staticBody , rigidBody et une Area. C'est L'area2D qui va m'intéresser car elle agit comme une zone qui détecte tout corps rentrant ou sortant.
voici donc ma plateforme de transition:



On a donc une area2D avec un sprite de Drapeau et une collisionShape qui définit la zone d'action de l'area2D.

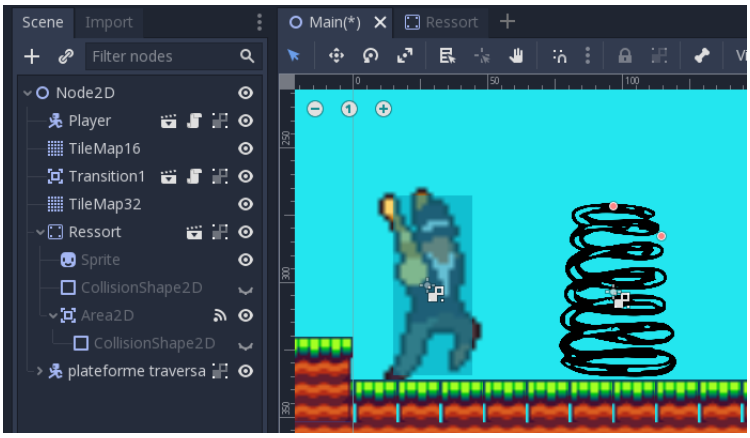
Un Nœud Area2D permet d'utiliser la fonction GetOverLappingBody() qui renvoie une liste de tout les corps en contact avec elle. Mon but était alors de vérifier si l'un de ces corps est le joueur, si oui alors on change de scenes pour passer à la prochaine.

```
public override void _Process(float delta)
  {
      var touch = GetOverlappingBodies();
          foreach (var guys in touch)
        {
                if (guys == "Player");
                {
                        GetTree().ChangeScene(nextScene);
```

```
                }
        }
}
```

2.3 Plateforme de saut

Enfin je me suis occupé d'une des plateformes les plus populaires des jeux de plateformes, celle qui nous éjecte vers le haut lorsqu'on lui saute dessus, pour cela il fallait là aussi que je détecte les collisions avec le joueur mais j'ai opté pour une solution différente (principalement pour étudier toutes les possibilités de godot). Voici à quoi ressemble cette plateforme :



elle est à l'origine un staticBody puisque c'est une plateforme qui ne va pas bouger mais elle possède aussi une area2d qui lui permet de détecter les collisions venant du dessus. Comme vous pouvez le voir sur l'image ci-dessus à coté du Noeud area2D de ressort il y a une image de signal, cela signifie que j'ai connecté cette area2d via l'interface de godot (ici elle est conectée au noeud joueur). cela m'a permis d'utiliser dans le script du joueur une nouvelle fonction la voici:

```
private void _on_Area2D_body_entered(Area2D body)
        {
        if (body.Name == "Player")
                {
                        motion.y = -500;
                }
        }
```

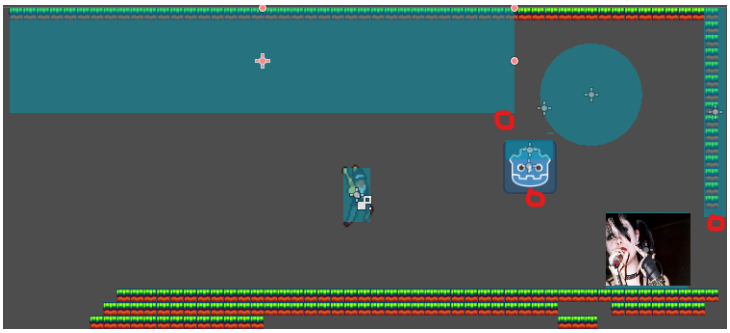
Maintenant à chaque fois qu'un nouveau corps rentre dans l'area2d connectée cette fonction sera appelée avec comme argument le corps concerné.
Maintenant il me suffit de vérifier si le corps en question est le joueur, si oui alors on modifie la vitesse du joueur sur l'axe y et c'est pour cela qu'il est intéressant d'avoir cette fonction directement dans le script du joueur car avoir accès à la

vitesse du joueur serait bien plus compliqué si ce n'était pas le cas.
Malgrès cela cette solution n'est pas parfaite car lorsqu'on sauvegarde Le Noeud ressort comme une scène (afin de pouvoir la dupliquer plus aisément dans d'autre niveau), La connection entre le joueur et L'area2d elle n'est pas sauvegardée, je dois donc reconnecter le joueur et L'area dans chaque nouveau niveau ce qui n'est as très pratique, c'est donc un autre point à améliorer pour la prochaine fois.

3 Level Design

3.1 Idée du 1er niveau

Comme évoqué dans le précédent rapport de soutenance, le premier niveau est essentiellement une énigme qui repose sur un système de balance entre un bloc léger et un bloc lourd. Le bloc lourd entraîne l'autre vers le haut tout en s'enfonçant vers le bas et en bloquant l'accès à la sortie du niveau. Afin de franchir cet obstacle, le joueur peut excercer son poids sur le bloc léger afin de le faire s'enfoncer progressivement, tandis que le bloc se lourd se met à monter. Voci un aperçu du rendu:



Les cercles rouges correspondent à l'emplacement des prises de grappin correspondantes après mise en commun. Le joueur aurait la possibilité d'exercer son poids sur le bloc léger de deux manières différentes: en s'appuyant dessus ou en attrapant la prise qui se situe en dessous. Cependant, à l'instant ou le personnage ne performe plus aucune de ces deux actions, le bloc lourd tombe trop rapidement pour permettre au joueur d'atteindre la sortie en courant. A la place, le joueur doit pouvoir utiliser son grappin pour atteindre la prise vers la sortie, mais il ne peut pas être simultanément accroché au bloc et libre de l'utiliser pour atteindre la sortie. Pour passer ce niveau, le joueur devrait remplir ces conditions dans le bon ordre:
- S'aggriper à la prise située en dessous du bloc léger afin de l'entraîner vers le bas.
- Utiliser le grappin sur la prise attachée au plafond avant que le bloc léger ne remonte totalement.
- Se laisser tomber au-dessus du bloc léger, ce qui va progressivement le refaire tomber et faire monter le bloc lourd en conséquence.
- Tout en restant sur le bloc léger, s'aggriper à la prise proche de la sortie.
- Félictations !

En sachant ça, il fallait mettre en place cette idée et fournir un code apte à dicter les mouvements de la balance entre les deux blocs.

3.2 Code de la balance

Voici le code qui dicte les mouvements du bloc léger:

```
public class Light_Block : KinematicBody2D
{
        const float gravity = 100.0f;
        public Vector2 velocity;
        public static bool falling;
        public static bool loose;
        public static bool stomp;

        public override void _PhysicsProcess(float delta)
        {
                loose =false;
                var motion = velocity * delta;
                if (stomp)
                {
                  velocity.y=0;
                  falling = true;
                  velocity.y += 10*delta * gravity;
                  MoveAndCollide(motion);
                }

                if (!stomp)
                {
                        falling = false;
                        velocity.y -=10*delta * gravity;
                        MoveAndCollide(motion);
                }

                if (IsOnFloor() && stomp)
                {
                        loose =true;
                        velocity.y=0;
                }
        }
}
```

Avant la mise en commun, je n'avais pas de personnage pour remplir les conditions qui mettaient mon bloc léger en mouvement. J'avais donc simulé sa présence par des touches du clavier distinctes qu'il suffisait de presser comme conditions pour chacun des "if". Désormais, mon champ de possibilité est plus restreint et je devais combler ces conditions autrement. Godot calcule la vitesse grâce à la valeur delta multiplié à la constante gravité qui s'ajoute à une valeur initialement nulle. "stomp" correspond à un booléen qui est vrai si le joueur s'appuie sur le bloc léger, qui descend vers le bas en conséquence, et inversement

lorsque le joueur quitte cette position et que stomp n'est plus vérifié.

Cependant, même si le bloc est visuellement immobile lorsque qu'on lance le jeu, il continuait à accumuler progressivement de la vitesse même dans une position statique, ce qui faisait qu'il pouvait mettre du temps à "répondre" aux différentes actions du joeur et le niveau n'aurait aucun sens dans ce cas. Par conséquent, j'ai ajouté une autre condition qui réinitialise la vitesse du bloc lorsqu'il est au sol et que le personnage se situe toujours au-dessus, de manière à ce qu'il remonte immédiatement lorsque le personnage quitte sa position. D'une manière similaire, j'avais besoin de faire en sorte que la vitesse reste nulle lorsque le bloc était entraîné au maximum vers le haut par le bloc plus lourd, mais je ne parvenais pas vraiment à trouver une condition valable qui ne créait pas d'autres problèmes. Par conséquent, j'ai décidé de faire en sorte que la vitesse du bloc léger alternerait entre un état nul et extrêmement rapide au moment où le personnage mettrait les pieds dessus. Le rendu donne une chute lente mais progressive de ce bloc censé être significativement plus léger que l'autre à tel point que le poids du personnage fasse à peine la différence, ce qui correspond assez bien à l'impression que je souhaitais donner.

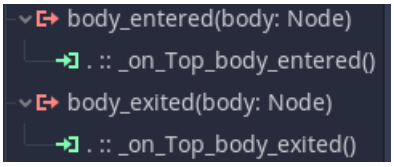
Maintenant, en ce qui concerne le bloc lourd:

```
public class Heavy_Block : KinematicBody2D
{
        const float rise = 100.0f;
        Vector2 traction;

        public override void _PhysicsProcess(float delta)
        {
                var move = traction*delta;
                if (Light_Block.falling)
                {
                        traction.y=0;
                        traction.y -= 7*delta*rise;
                        MoveAndCollide(move);
                }

                if (Light_Block.loose)
                {
                        traction.y=0;
                }

                if (!Light_Block.falling)
                {
                        traction.y +=10*delta*rise;
                        MoveAndCollide(move);
                }
```

Les mouvements de ce bloc sont totalement régis par ceux du bloc léger, d'où les différents booléens dans le script de ce dernier qui permettent d'activer la montée et la descente du bloc lourd, qui va dans la direction opposée à celle du bloc léger. La gestion de sa vitesse suit un raisonnement similaire. En effet, elle n'est pas toujours identique à celle du bloc léger, car bien que ça puisse être incohérent, je voulais faire en sorte que les deux blocs atteignent leur états statiques à peu près simultanément.

3.3 Influence du personnage

Le booléen "stomp" n'est affecté par aucun des deux scripts précédents, car il dépend de la position du personnage par rapport au bloc léger. Afin de provoquer cette réaction, j'ai implémenté une Area2D sur le haut du bloc léger et lui ai assimilé deux signaux:



Le code associé à cette Area2d est le suivant:

```
 public class Top : Area2D
{
   public void _on_Top_body_entered(Area2D body)
   {
        Light_Block.stomp=false;
          if (body.GetName() == "Character")
                {
                        Light_Block.stomp =true;
                }
   }

   public void _on_Top_body_exited(Area2D body)
   {
        if (body.GetName() == "Character")
                {
                        Light_Block.stomp =false;
                }
   }
}
```

Ces signaux détectent si le joueur se situe le haut du bloc léger ou non, respectivement. La première condition à elle seule ne suffisait pas car contraitementg aux Input de touches du clavier, cette fonction est "définitive" dans le sens où il suffisait au joueur d'effleurer la surface recherchée du bloc léger pour

que celui entre dans un état de chute perpétuel, d'où la fonction "exited" qui remarque quand le joueur ne se trouve plus sur le bloc.

Par ailleurs, quelques mises à l'échelle ont été faites en prenant en compte la vitesse du joueur, des blocs, et de la distance entre ces derniers. Elle est cependant encore perfectible.

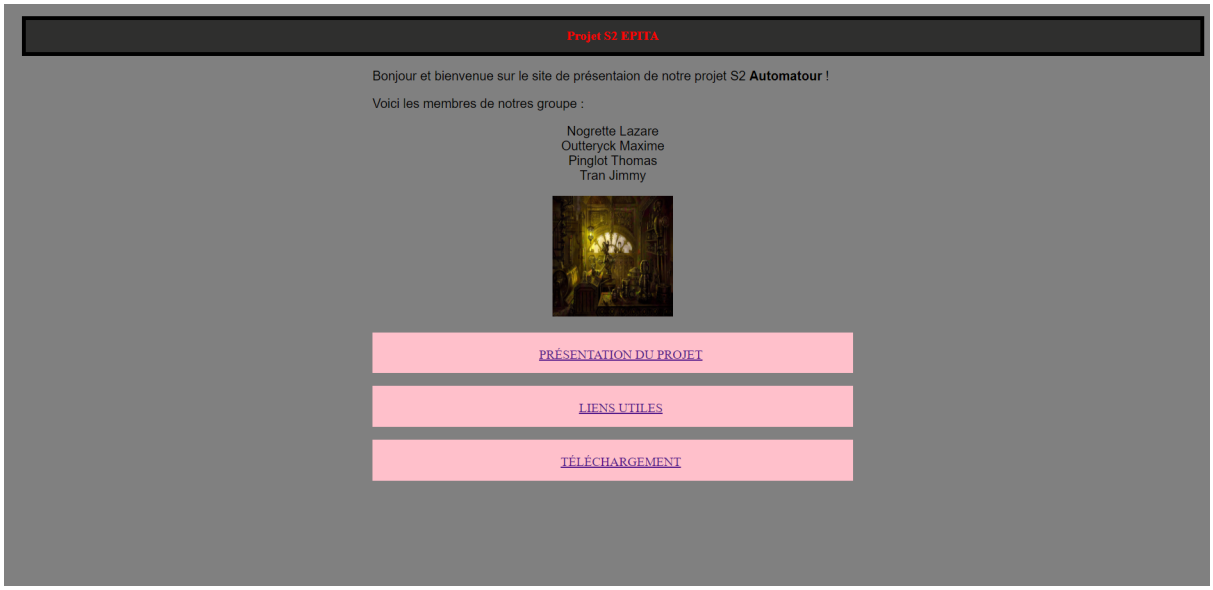
3.4 Objectifs

Le premier niveau est à peu près achevé, mais j'aurais aimé pouvoir offrir un prototype du deuxième mais je me suis heurté à trop de difficultés pour fournir une bonne idée de ce à quoi il ressemblerait. L'inspiration a mis également du temps à venir et à s'éclaircir...

4 Site web

4.1 Html et CSS

Pour la réalisation de notre site web, j'ai utilisé les langages HTML et CSS avec l'éditeur de code SublimeText. L'HTML permet de créer et structurer la page web, le CSS lui permet de mettre en forme le site en modifiant les couleurs, la police de caractère, la taille, et tout autre chose de mise en page.
Notre Site web se présente comme ceci :



Sur cette page d'accueil on peut y retrouver des liens vers des pages de téléchargement de fichier, nottament ce rapport mais aussi a la fin de notre projet un lien permettant de télécharger le jeu fini.
Il y a aussi un lien vers la présentaion du projet qui sera complété par la suite où il se trouvera les membres, la chronologie de réalisation, les problèmes rencontrés, les solutions envisagées.
Pour finir, le troisième lien emmène vers une page avec divers lien de site, application que nous avons utilisé durant notre projet tel que godot engine et github.

Pour la mise en page, j'ai agrandi la taille des caractère grâce a la balise

```
<FONT size="6">
```

Dans le fichier CSS, j'ai créer une balises permettant d'afficher des bande de couleur (ici en haut de page avec le titre a l'interieur et pour encadrer les differents lien vers les autres sites) mais aussi une autre pour changer de couleur le fond de la page, pour centrer le text, pour positionner le bouton de retour a

l'accueil. Tout ces balise était codable dans le fichier html directement, mais en les mettant dans un fichier CSS cela permet de les reutiliser plus facilement sur toute les pages, et ainsi avoir la meme mise en page partout.

```
<div id="fond">
            <FONT size="6">
                <p>
                    Bonjour et bienvenue sur le site de
                    presentaion de notre projet S2
                    <strong>Automatour</strong> !
                </p>
            </FONT>
</div>
```

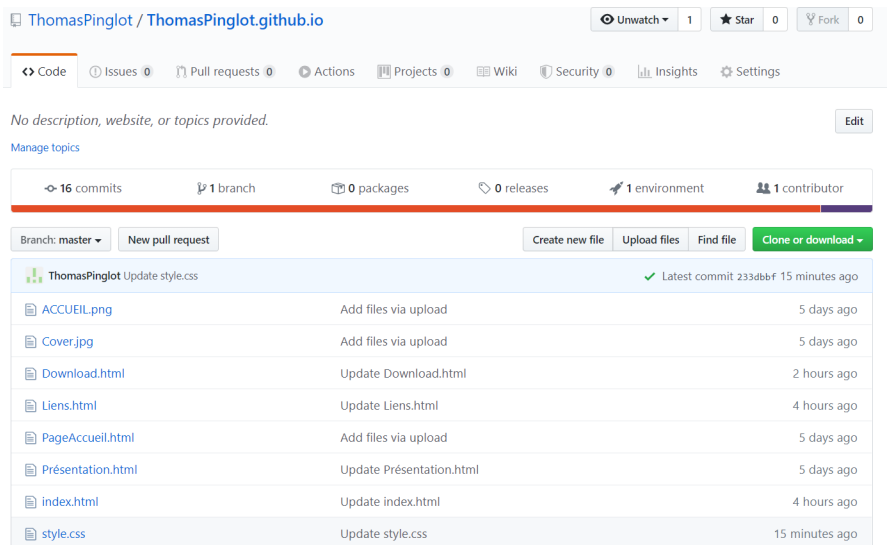
Dans cette exemple, la balise "fond" va permettre de structurer tout ce qui va se trouver dans cette balise , c'est à dire elle va créer une zone, ici rectangulaire, avec des espacements sur les cotés

Dans la balise "fond" du fichier CSS on va définir la taille en largeur de notre rectangle avec "width: la taille en pixel;"et la proportion de l'espace à gauche et à droite de ce rectangle avec "margin:0 auto;" qui permet donc ici de mettre le même espacement à gauche et à droite.

```
#fond
{
        width: 1200px;
        margin:0 auto;
        align-content: center;
}
```

4.2 Hébergement du site

Afin que notre site soit accessible par tout le monde via un lien, il fallait trouver un moyen de l'héberger.Pendant un long moment de recherche j'ai cru qu'il allait falloir acheter un nom de domaine et un serveur pour notre site et que sans cela on allait pas pouvoir l'héberger. Il y a avait quelque sites qui proposaient d'héberger gratuitement en envoyant un mail et sous certaine conditions, mais a chaque fois il y avait un delai d'attente de plusieur mois et il nous faillait l'héberger maintenant.
C'est alors que je trouvasse une vidéo d'une personne expliquant que github permet d'heberger gratuitement et en seulement quelques minutes notre site en créant un git repository avec tout notre code HTML.

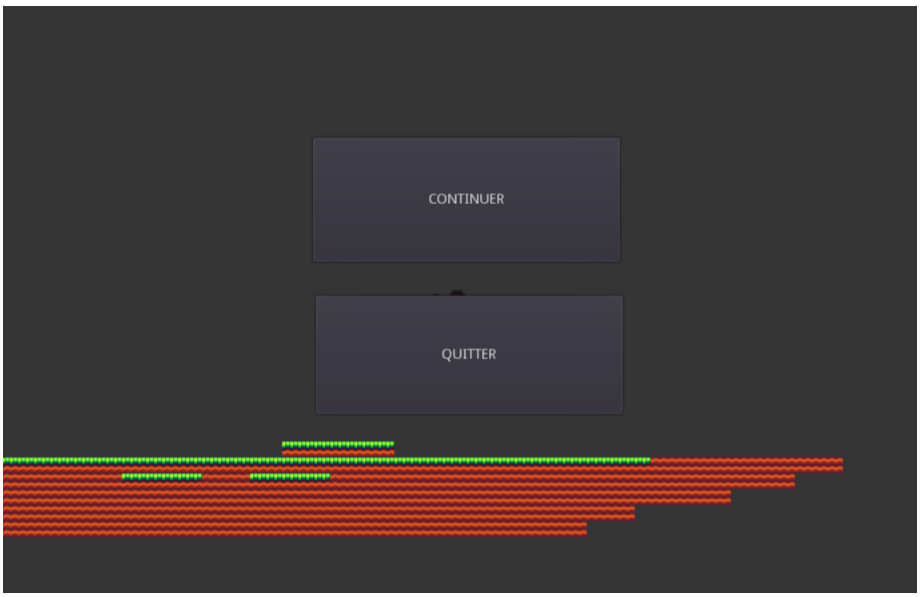


Néanmoins, on ne peut pas choisir son nom de domaine si on ne l'a pas préalablement acheté, le lien du site aura donc le nom d'utilisateur git de celui qui l'a créé suivi de ".github.io".

5 Menu

5.1 MenuPause

La derniere fois le menu n'était connécter a aucune vrai scene de jeu, maintenant je l'ai connécté avec une scene de jeu avec des animations et control de mouvement. Et c'est a ce moment que je me suis rendu compte que le menu pause actuel ne marchait pas vraiment. En effet, on pouvait bien acceder a ce menu pause en appuyant sur echape et relancer le jeu avec le bouton continuer, mais en relancant le jeu, le changement de scene faisait que on retourner au tout debut. J'ai donc décidé de changer de méthode pour ce menu en optant pour l'affichage d'un rectangle de couleur noir/transparent et de bouton "continuer" et "quitter" lorsqu'on appuie sur la touche échape.



Le jeu se met en pause et affiche cela, avec soit la possibilité de quitter et de revenir au menu principal, soit de continuer en enlevant ces bouton, cette assombrissement et de remettre le jeu en action. De cette manière, le joueur pourra voir où il s'était arrété en arrière plan et ne pas être surprise en reprenant le cours du jeu.

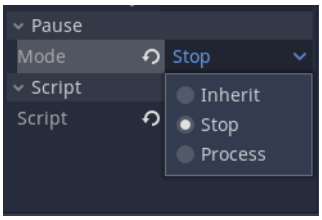
Pour faire cela, jai utilisé la fonction :

```
GetTree().Paused == true;
```

Lorsqu'elle est sur "true", le jeu s'arrête, et si elle repasse sur false le jeu se remet en normalement.
Grâce a cela, on peut créer une boucle permettant de mettre le jeu en pause et d'afficher ce que l'on veut en appuyant sur échap.

```
if (Input.IsActionJustPressed("ui_cancel"))
        {
                        if (GetTree().Paused == false)
                        {
                jouerButton.Show();
                                quitterButton.Show();
                BlackOverlay.Show();
                GetTree().Paused = true;
                        }
                 }
```

Un des plus gros problème que j'ai rencontré sur ce menu pause, c'est le fonctionnement des noeuds pendant la pause. En effet, les noueds peuvent être affécté à trois catégories pendant la pause : hériter,arrêter et processus. Ils étaient tous sur hériter de base, ils avaient donc tous la propriété de leur parents, et lorsque j'activé la pause, ils étaient tous affécté de telle sorte a ce qu'on ne puisse plus rien faire.Tout était figé et je ne comprenais pas pourquoi, c'est avec des recherches sur le site de GodotDocs que j'ai finalement compris que mon problème venais de là.



J'ai donc mis tout ce qui devait pouvoir être activé en "process", cela permet de traiter le noeud meme s'il est en pause, et le personnage en mouvement sur "stop" pour qu'il soit en pause quoi quil arrive si le jeu se met en pause.

6 Grappin

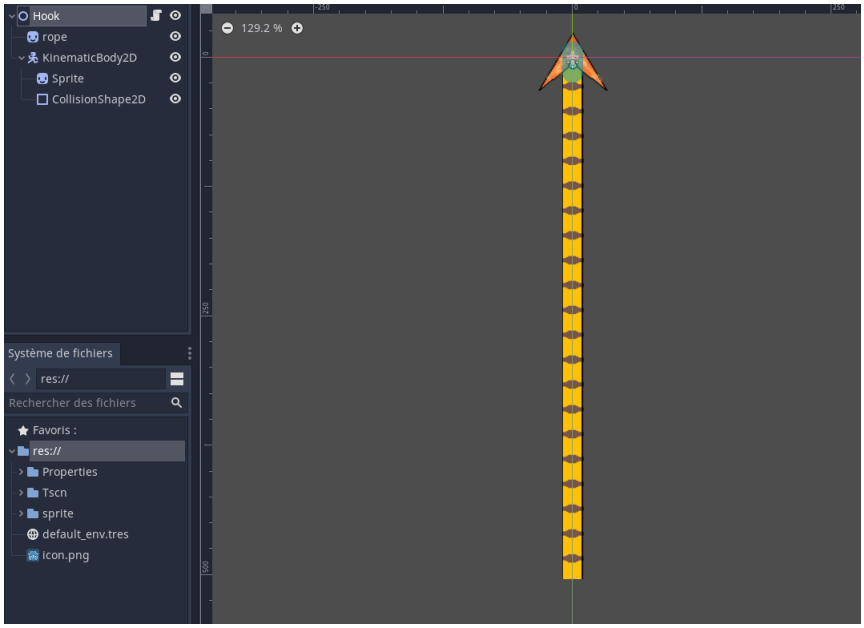
6.1 Réalisation

Pour la première version du grappin, un des problèmes majeurs était la mise en place de la corde (Line2D) qui restait encore une méthode très naïve et simple, dans le sens où elle peut être visuellement meilleure. De plus,la mise en place du grappin se basant sur la node "raycast2D" qui avait l'avantage de pouvoir gèrer la direction et la portée du grappin posait des limites quant à la conception du grappin.

L'objectif était donc de fixer ces problèmes tout en gardant le grappin fonctionnel. Dans ce modèle de grappin, le bloc est séparé en 2 grands blocs.

6.1.1 Méchanisme du grappin

Le but dans cette partie était de gérer le fonctionnement du grappin dans son entiereté, en incluant son déclenchement, son orientation, les collisions avec le décor de la scène ainsi que tout l'ensemble lié au visuel tel que la corde et l'apparition du grappin lors de son utilisation.



2 fonctions essentielles sont présent: "_Process" et "_PhysicsProcess":

- "_Process" est une fonction gère le visuel du grappin tel que sa visibilité et l'orientaion du grappin et de la corde lors du déclenchement.

```
    public override void _Process(float delta)
    {
        Visible = in_air || hooked;
        // Si un des booleens est vrai, alors le
        //grappin sera visible.
        if (!Visible)
        // Dans le cas contraire, il ne renvoie
        // rien.
        {
            return;
        }
         rope.RegionRect = new Rect2(0,0,21,
         cur_goal.Length());
         //prolongement la longueur de la corde lors
         // de l'activation du grappin.
         //cur_goal: vecteur partant de la position
         //du joueur, allant jusqu'a la position
         //du grappin.
    }
```

- "_PhysicsProcess" va prendre manipuler les mouvements du grappin du grappin, en détectant les collisions avec les tuiles du jeu.

```
    public override void _PhysicsProcess(float delta)
    {
        arr.GlobalPosition = goal;

        if (in_air)
        {
            var colliison =arr.MoveAndCollide(direction*speed);

            if (colliison != null)// Verifie si une
            //collision a eu lieu.
            {
                hooked = true;
                in_air = false;
            }
        }

        goal = arr.GlobalPosition;
    }
```

Il est important de comprendre la raison de l'emploi de ces 2 différents fonction. D'un côté, "_Process" a pour particularité d'exécuter chaque frame aussi tôt que possible, permettant de faire apparaître les choses de manière fluide. Quant à "_PhysicsProcess", celle-ci éxécute les frames à un intervalle de temps régulier, ce qui permet d'effectuer des tâches qui ont besoin d'être constant comme les déplacements d'objet.

2 autres fonctions sont également contenues dans ce bloc: shoot qui en quelque sorte raffraîchit la position du grappin et permet l'utilisation du grappin et release qui change la valeur des valeurs "in_air" et "hooked", il permet de rendre alors le grappin invisible lorsque l'on ne l'utilise pas.

6.1.2 Déplacement du joueur

Cette partie est intégrée dans le script du "Player" puisqu'il intéragit avec ses déplacements. Il va d'une part gérer l'input lié au déclenchement du grappin.

```
public override void _Input(InputEvent @event)
    {
        if (@event is InputEventMouseButton eventMouseButton)
        {
            if (@event.IsPressed())
            {
                hook.shoot(eventMouseButton.Position
                - GetViewport().Size*0.5f);
                //Appelle fonction shoot pour recuperer
                //la position de la souris
            }
            else
            {
                hook.release();
            }
        }
    }
```

Enfin, pour pouvoir effectuer la traction du joueur vers le grappin, il suffira de presser à flèche du haut.

```
if (hook.hooked && Input.IsActionPressed("ui_up"))
```

6.2 Tâches accomplies et à accomplir

Dans cette partie, le travail le plus important était la conception d'un grappin fonctionnel, avec un aperçu visuel correct(animation du lancement). Les prochains objectif à réaliser seraient la mise en place d'un système qui forcerait le grappin à pouvoir s'accrocher uniquement sur des tuiles spécifiques et éventuellement implémenter une portée au grappin, pour éviter qu'on puisse traverser le niveau d'un seul coup de grappin.

7 Conclusion

La mise en commun totale a pris un peu de retard mais nous avons tous bien avancé dans nos parties respective, Le grappin marche très bien, de nouvelles plateformes ont été créer, la première énigme fonctionne avec le personnage et nous avons un site qui fonctionne.

Mais le travail n'est pas fini pour autant il reste pas mal de chose à faire, le grappin doit offrir plus de possibilité comme ramener un objet ou permettre de se balancer, le nombre de différentes plateformes est encore trop faible est doit être augmenter, il n'y a pas assez d'éléments d'énigmes et la première n'a pas encore été testé avec le grappin et enfin le site internet est encore trop vide et ne demande qu'à être complété, de plus des sauvegardes vont devoir être créée.